TP 5 : Reconnaissance de mots

L'intégralité du TP est à réaliser en `OCaml`.

Préliminaires

Dans ce TP, on cherche à étudier le problème suivant : Si M est un mot et T un texte, M est il présent dans T ? Si on reformule cela à l'aide d'expression régulière, cela revient à se demander si T appartient au langage caractérisé par $\Sigma^\star m \Sigma^\star$. On sait donc qu'il est possible de construire un automate A pour répondre à cette question. On étudie dans ce sujet une méthode qui permet de construire un automate minimal en nombre d'états pour répondre à cette question.

Comme dans le TP précédent, on considère que Σ est de la forme $[\![0, n]\!]$ pour des raisons de simplicité.

On pose $T = t_1 \dots t_n$ et $M = m_1 \dots m_k$, et pour $1 \leq i \leq k$. De plus, on définit $M_i = m_1 \dots m_i$ le préfixe de M de longueur i. On pose aussi par convention $M_0 = \epsilon$.

L'objectif est de résoudre ce problème en temps linéaire en $O(n)$ avec un pré-traitement de complexité polynomiale en $O(k)$. Comme k sera souvent petit devant n, on aura ainsi la garantie d'avoir un algorithme efficace.

Manipulation d'automates

Pour manipuler les automates en `OCaml`, on introduit le type construit suivant :

```
type automate = { taille_sigma : int;
                  taille_q : int ;
                  q_0 : int;
                  finaux : bool array;
                  delta : int array array}
```

Ici, Σ et Q sont représentés par leurs tailles respectives. Si un état i est final, alors la i-ième cas de `finaux` vaudra `true`. Enfin, `delta` est une matrice telle que $\delta(q, a)$ soit stocké à la position `delta.(q).(a)`.

On représentera un mot par un tableau d'entiers donc un `int array`.

Question 1 : Écrire une fonction `delta_star : automate -> int array -> int -> int` qui, étant donné un automate A, un mot x et un entier i, renvoie l'état $\delta^\star(q_0, x_1 x_2 \dots x_i)$.

Question 2 : En déduire une fonction `reconnu : automate -> int array -> bool` qui, étant donné un automate A et un mot x, teste si x est reconnu par A.

Automates des préfixes

Si $u, v \in \Sigma^*$, on note $u \sqsubset v$ si u est suffixe de v. On note aussi

$$\sigma(u) = \max\{i : M_i \sqsubset u\}$$

c'est-à-dire la taille du plus grand préfixe de M qui est également suffixe de u.

Soit $A = (Q, \Sigma, I, F, \delta)$ l'automate déterministe complet défini par :

- $Q = \{0, \dots, k\}$;
- $I = \{0\}$;
- $F = \{k\}$;
- pour tout $q \in Q$ et $a \in \Sigma$, $\delta(q, a) = \sigma(M_q a)$.

Question 3 : Dessiner l'automate associé au mot $M = 012$ sur l'alphabet $\{0,1,2\}$.

Question 4 : Écrire une fonction `automate_vide : int -> int -> automate` qui, étant données une taille de Q et une taille de Σ, renvoie un automate dont toutes les transitions sont initialisées à -1, où 0 est l'état initial et $|Q|-1$ est le seul état final.

Question 5 : Écrire une fonction `sigma : int array -> int array -> int` qui, étant donnés un mot M et un mot u, calcule $\sigma(u)$.

Question 6 : Écrire une fonction `automate_suffixe : int array -> automate` qui, étant donné un mot M, renvoie l'automate des suffixes proposé plus haut.

Question 7 : Cet automate est dans l'état k après avoir lu x depuis 0 si et seulement si x a pour suffixe M. En déduire une fonction `automate_facteur : int array -> automate` qui, étant donné un mot M, renvoie un automate reconnaissant le langage des mots contenant M.

On remarque que σ vérifient des propriétés intéressantes :

Soient $u, v \in \Sigma^*$ et $a \in \Sigma$.

1. Si $u \sqsubset v$, alors $\sigma(u) \leq \sigma(v)$.
2. $\sigma(ua) \leq \sigma(u)+1$.
3. $\sigma(ua) = \sigma(M_{\sigma(u)}a)$

Question 8 : Modifier les fonctions précédentes pour exploiter cela et calculer plus efficacement σ.

Question 9 : En déduire une fonction `est_facteur : int array -> int array -> bool` qui, étant donnés un texte T et un mot M, teste si M est un facteur de T.